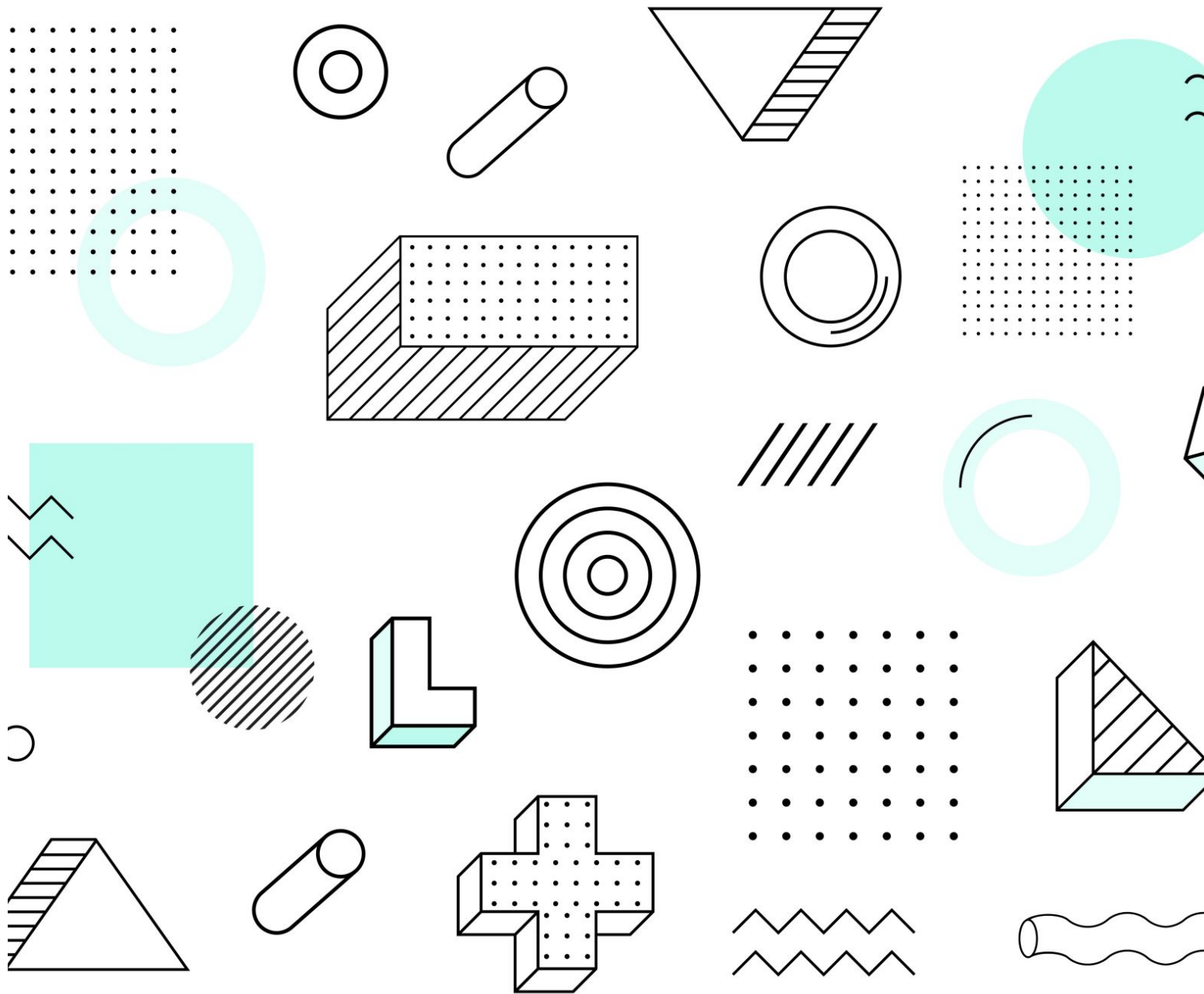


Alocação de Memória

First-fit, Next-fit, Best-fit e Worst-fit

Otávio Garcia Capobianco
NUSP: 15482671



Introdução

- Neste EP, implementamos um simulador de algoritmos de alocação memória, em que arquivos .pgm são manipulados de modo que, a nível abstrato, representa alguns dos aspectos de como o sistema operacional gerencia a memória do computador
- Os algoritmos escolhidos foram **First-fit, Next-fit, Best-fit e Worst-fit**
- Com nosso programa, partindo de um entendimento decente de como tais algoritmos funcionam, podemos realizar experimentos que revelem como essa base teórica se manifesta em situações mais práticas
- Portanto, nesta apresentação abordaremos **pontos positivos e negativos** de cada um desses algoritmos que são evidenciados por meio desses experimentos realizados, bem como discutir alguns **detalhes da implementação** que permitam compreender os possíveis tradeoffs e recursos adicionais necessários para cada um deles

Especificações

- Os experimentos aqui detalhados foram realizados num computador com:
 - Processador **Intel Core i5-5350U** com 2 núcleos físicos e 4 núcleos lógicos
 - 8GB de memória RAM DDR3
- O programa supõe um sistema operacional **Linux**, e a distro utilizada durante a execução dos programas foi o Ubuntu (v24.04.2)
- Todo o código do simulador implementado foi escrito em C, e os gráficos presentes nesta apresentação foram gerados em Python, utilizando Matplotlib

Variáveis Auxiliares

- Dado que o programa simula situações de gerenciamento de memória, faz sentido ser cauteloso quanto ao **consumo de memória dos próprios algoritmos**
- Por isso, abordaremos as **variáveis adicionais** que cada um necessitou para funcionar. As que todos eles precisaram utilizar foram:
 - Variáveis para calcular a posição correta do arquivo .pgm a ser lida/escrita, e buffer de leitura
 - Um inteiro para registrar o tamanho do atual bloco contínuo de unidades livres, e outro para marcar o início do mesmo (para sabermos quando há espaço suficiente)
 - Um inteiro i para iterar sobre todas as posições do .pgm
- Fora essas, cada algoritmo precisou especificamente de:
 - **First-fit: nada** adicional
 - **Next-fit:** inteiro que registra a **última unidade alocada pelo algoritmo**
 - **Best-fit e Worst-fit:** inteiro que registra o tamanho do **melhor bloco contínuo de unidades livres** que satisfaça as unidades requisitadas (o menor no best, o maior no worst) e outro para marcar o início do mesmo

Experimentos – Tempo de Execução

- Avaliaremos os algoritmos em dois aspectos principais: tempo de execução e quantia de alocações não concretizadas (falhas)
- **O first-fit e o next-fit** se destacam pela sua eficiência em **tempo de execução**, além de necessitarem de ligeiramente menos variáveis adicionais.
 - Isso pois o best-fit e o worst-fit necessariamente **varrem a memória inteira** em toda tentativa de alocação, o que é custoso. Os outros dois, embora no pior caso (sem espaço livre) necessitem percorrer toda a memória também, em média não precisam fazê-lo
 - O fator diferencial do next-fit, registrar a última posição alocada, tenta **evitar buscas desnecessárias que o first-fit tende a fazer**, e portanto esperamos que na maioria dos casos o next seja o mais rápido. Um aspecto negativo dessa abordagem do next é que ele em geral fragmenta a memória em pontos distintos, distribuindo espaços que podem ser inúteis, enquanto que o first, por sempre tentar as posições iniciais, tende a concentrar a alocação, deixando a região ocupada mais densa (na parte inicial da memória) e portanto a livre menos fragmentada, o que muitas vezes reduz o número de falhas

Experimentos – Falhas

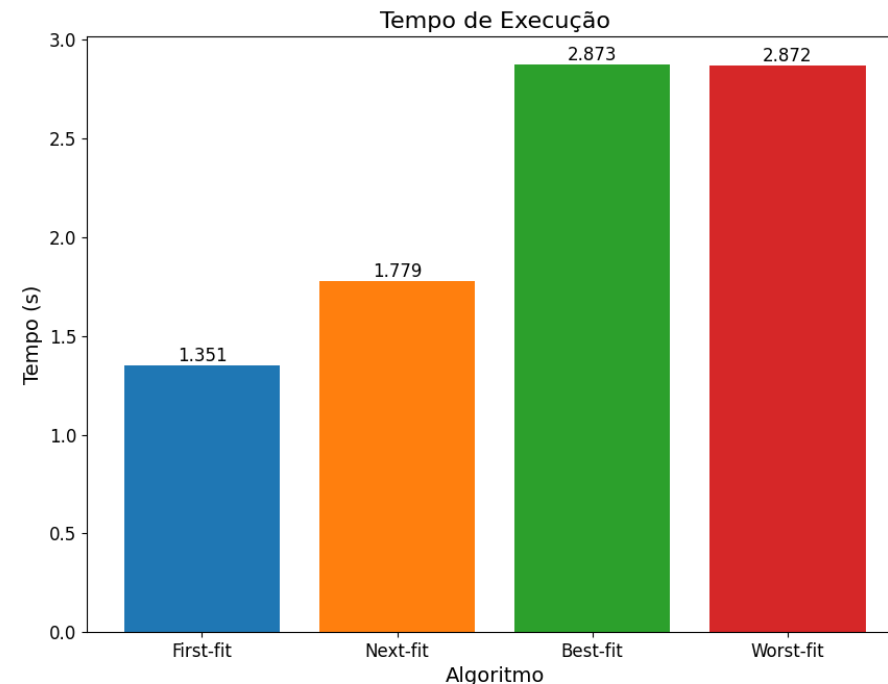
- O **best-fit** e o **worst-fit** se destacam por, em geral, serem **capazes de alocar mais memória**, por terem uma lógica um pouco mais complicada por trás de suas escolhas
 - Vale notar que, conhecendo a lógica por trás de cada um deles, é possível elaborar arquivos de trace que **favoreçam um em detrimento do outro**, dado que suas lógicas são, em grande parte, antagônicas
 - Para o first/next-fit, embora, como já mencionado, ao longo de muitas entradas diferentes espera-se que o first resulte em menos falhas, provavelmente não haverá uma diferença significativa o suficiente com apenas 4 arquivos de trace para traçar conclusões definitivas. Ainda assim, esperamos mais falhas destes dois em comparação ao best/worst-fit
- Tendo isso em mente, nos slides a seguir mostraremos os resultados de tempo e falhas dos experimentos realizados com cada arquivo de trace. Note que daremos mais ênfase à **comparação de tempo entre first e next**, nos seus respectivos trace, e **falhas entre best e worst**, mas mostraremos todos os dados obtidos para permitir uma noção mais ampla dos resultados

Experimentos – Considerações

- Todos os resultados expostos a seguir surgiram de execuções tomando como entrada o arquivo ep3-exemplo01.pgm, com os respectivos traces mencionados em cada slide
- Durante as execuções do programa, apenas o terminal que o estava rodando e o próprio SO estavam ativos, para fins de reprodutibilidade e coerência
- Como o programa é totalmente determinístico, existe pouca variação no tempo de execução, de modo que faz sentido omitir a variância e intervalo de confiança. Os valores apresentados nos gráficos de tempo de execução são as médias de 5 execuções distintas

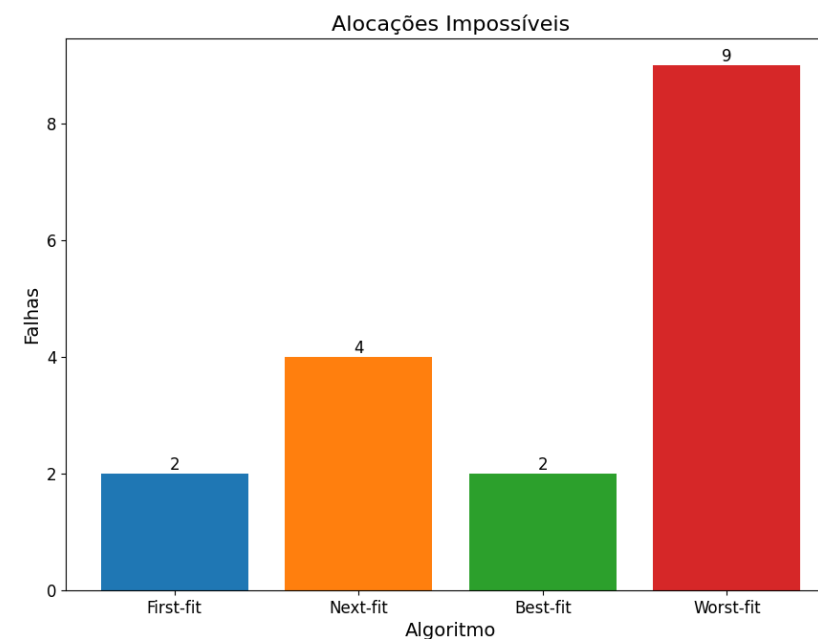
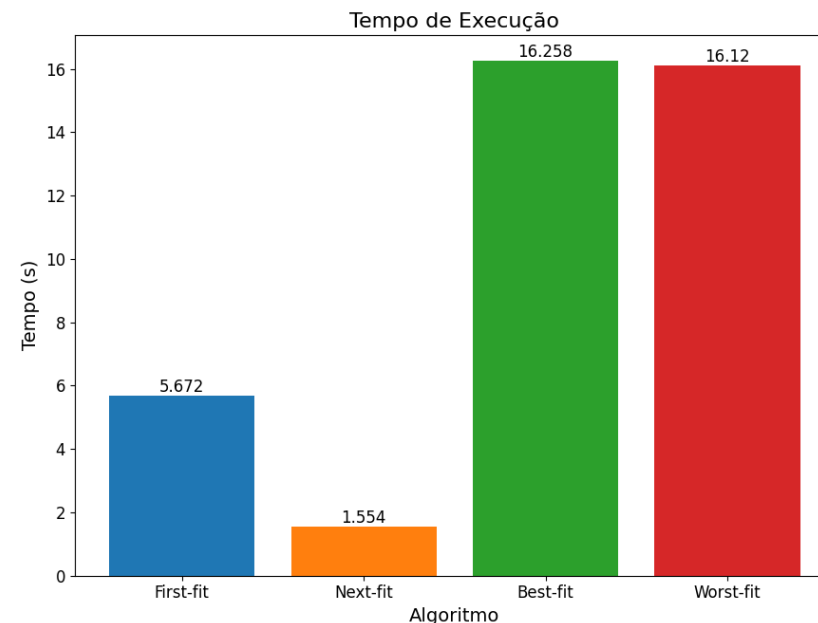
Trace First-Fit

- Aqui, utilizamos um exemplo mais minimal, de modo que **não houve falha** em nenhum dos algoritmos. Assim, best-fit e worst-fit foram praticamente idênticos em tempo de execução
- Criamos situações em que o **next-fit** precisou dar **quase uma volta completa** muitas vezes, primeiro escolhendo um valor só disponível no fim do pgm, depois um pouco antes, e assim por diante. Enquanto o next dava voltas grandes, o espaço livre ficava cada vez mais próximo do first-fit
- Escolher uma situação em que o first é mais rápido envolve, normalmente, casos mais específicos como este, e ainda assim houve um desempenho apenas 25% melhor



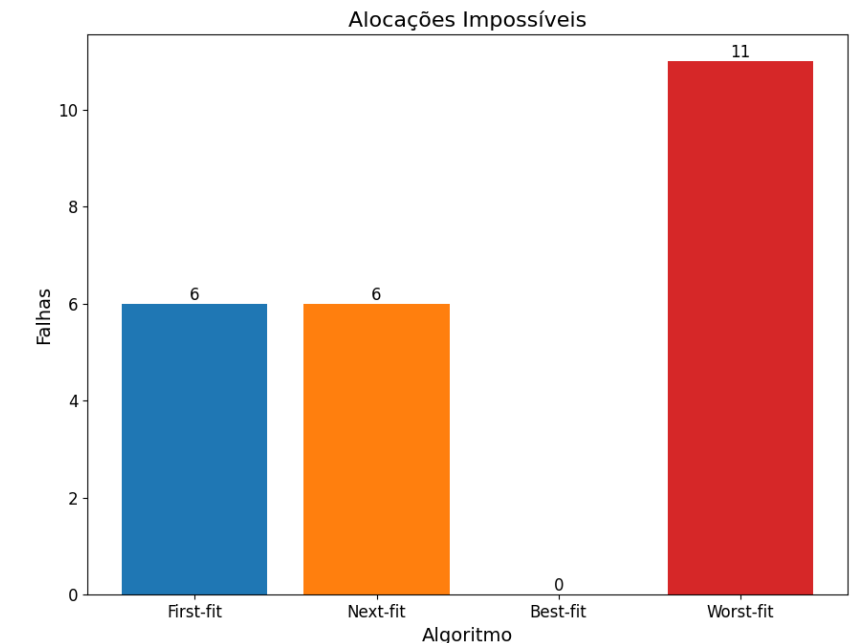
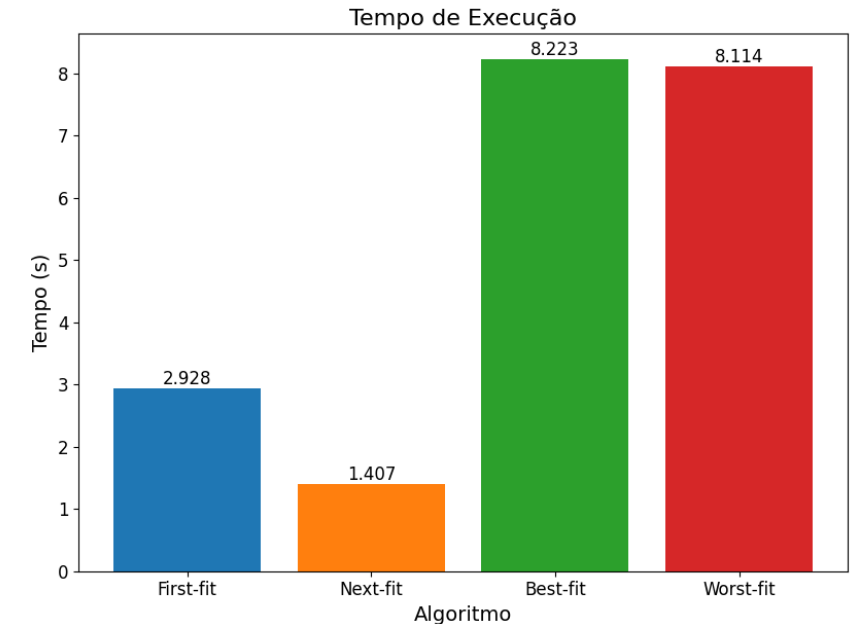
Trace Next-fit

- Este trace é maximal, requisitando toda a memória disponível
- Para melhor desempenho do **next-fit**, garantimos que a primeira metade do trace, que possui as maiores requisições, poderia ser **concluída em uma única volta** na memória pelo next. Para tal, sempre haviam espaços disponíveis sequencialmente, o que favorece o next e torna o first bem menos eficiente
- O next-fit foi **72% mais rápido** que o first-fit, mas este teve algumas falhas a menos
- Em questão de falhas, o ponto inesperado é o baixo desempenho do worst. Isso ocorreu pois os maiores espaços foram ocupados muito rapidamente, gerando falhas na primeira metade do arquivo, de valores maiores que supunham espaço grande livre



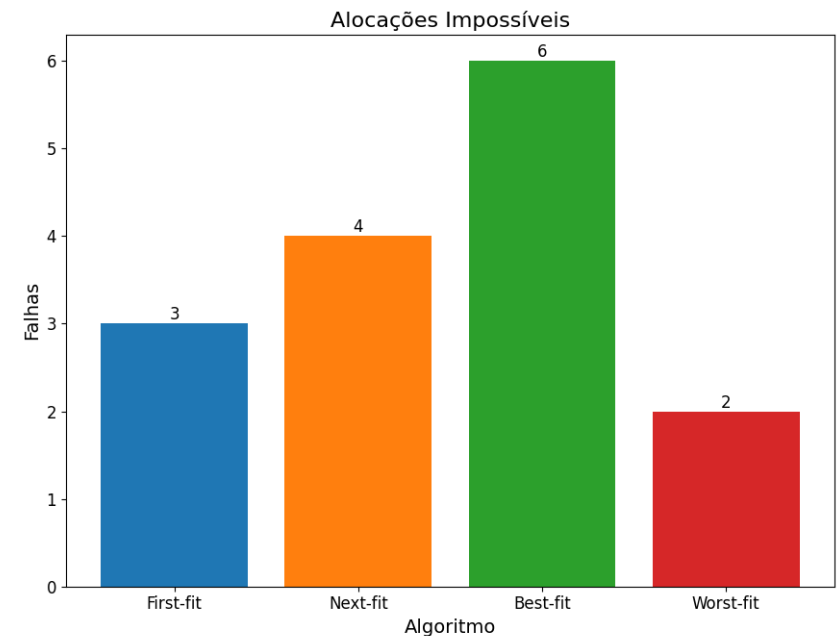
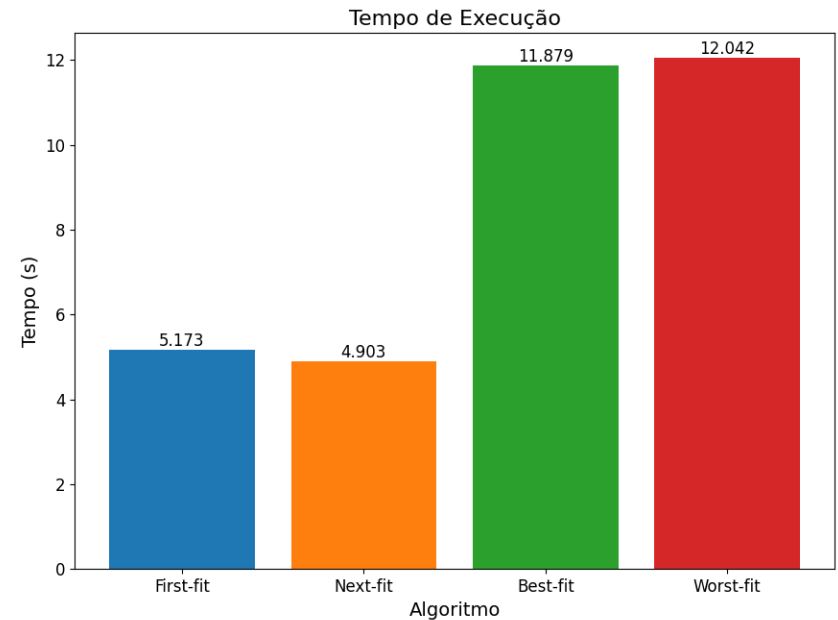
Trace Best-fit

- Este trace faz requisições de todos os espaços livres, no valor total de cada espaço, na ordem crescente de tamanho
- Essa abordagem garante que o **best-fit** sempre escolha um espaço de modo que, após a alocação, **não sobre espaço livre** nele. É o **cenário ideal** suposto pela premissa do best-fit
- A partir de uma certa linha do arquivo, o **worst-fit** não conseguiu fazer mais **nenhuma alocação**, dado que sua premissa, de guardar espaço futuro para requisições pequenas/médias, é violada, tendo em vista que nunca haverá uma requisição menor do que a atual, apenas maiores ou iguais, para as quais já não haverá espaço disponível



Trace Worst-fit

- Este trace tem uma **etapa inicial** em que cria um **cenário ideal para o worst-fit**: algumas requisições menores são feitas, e logo em seguida pede-se um espaço grande que, somado a essas menores, totaliza o maior espaço da memória inicialmente
- Ao mesmo tempo, pelo **best-fit**, essas requisições menores espalham **espaços livres pequenos demais** pela memória. Ao fim do trace, tais espaços são requisitados, e enquanto o worst-fit os têm disponíveis, o best os fragmentou, e ocorre falha neste último
- Logo, enquanto o worst-fit tem aqui em parte um cenário ideal, a **hipótese do best-fit**, de deixar grandes espaços para grandes requisições, **não é tão aproveitada**, e os espaços fragmentados fazem mais diferença



Conclusão

- Pelos últimos slides, é visível que, conhecendo os algoritmos, podemos elaborar circunstâncias que configurem cenários ideais ou ruins, e em situações reais qualquer um dos dois pode acabar ocorrendo
- Por conta disso, **nenhum dos algoritmos é universalmente melhor**, e é conveniente escolher aquele que, **em média, melhor atende o propósito da aplicação**, seja este o tempo de execução, quantidade de requisições atendidas, ou um meio termo entre os dois
- Em grande parte, mesmo com uma quantidade baixa de arquivos de teste, pudemos observar uma **efetivação na prática do que prevíamos pela teoria**
- De modo geral, pelos testes apresentados e overheads discutidos para cada algoritmo, é possível ter uma noção bem estruturada sobre os **tradeoffs** que devem ser considerados, bem como algumas das razões que fazem com que estes ocorram